Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования «Кубанский государственный университет»
(ФГБОУ ВО «КубГУ»)

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

по выполнению лабораторных работ по дисциплине
«Автоматизация тестирования программного обеспечения»
для бакалавров направлений подготовки 01.03.02 «Прикладная математика и информатика» 02.03.03 «Математическое обеспечение и администрирование информационных систем» профиль «Технология программирования», 09.03.03 «Прикладная информатика» профиль «Прикладная информатика в экономике»

Краснодар
2026

Таблица 1. План лабораторных работ

№	*Тема лабораторной работы*	*Срок выполнения (колво пар)*
1	Примитивный поиск дефектов в ПО. Тестирование формы входа	1
2	Построение баг-репорта	1
3	Чек-лист. Построение классов эквивалентности. Тестирование граничных значений	1
4	Тестирование таблиц решений. Попарное тестирование	2
5	Комбинаторное тестирование. Диаграмма состояний и переходов	2
6	Сценарное и исследовательское тестирование	1
7	Отчеты о тестировании	2
8	Чек-лист для тестирования веб-форм	1
9	Тестирование строки поиска	1
10	Комплексное тестирование программного продукта.	2

Требования к содержанию отчетов о выполнении лабораторных работ

В процессе изучения курса «Автоматизация тестирования программного обеспечения» должны быть подготовлены и сданы десять отчетов о выполнении лабораторных работ (см. таблицу 1). Лабораторные работы выполняются в соответствии с заданием.

Отчет должен содержать заголовок, тему лабораторной работы, задание, описание хода выполнения работы, необходимые прикладные материалы (схемы, скриншоты и т.п.) и выводы по работе.

Отчет сдается в электронном виде на паре в установленные сроки.

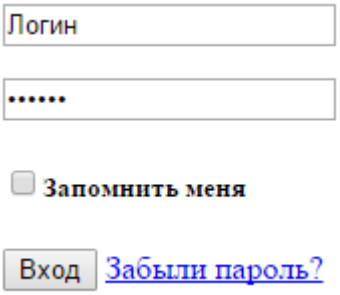
Лабораторная работа 1

Примитивный поиск дефектов в ПО

Задание 1. Написать программу, определяющую тип треугольника по трем его сторонам. Протестировать всевозможные дефекты разработанной другим студентом группы программы по заданию 1. Оформить тесты с пометками выполнено \ не выполнено.

Тестирование формы входа

Рассмотрим часто встречающуюся форму с двумя полями: логин и пароль.



Под логином может быть email, номер телефона или ник.

— найти форму входа должно быть легко

Она должна быть на виду или доступна в один клик. Пользователь должен сразу понять, что этот сайт имеет форму входа. Иначе пользователь просто не поймет, или пропустит и не зарегистрируется. Тогда мы потеряем потенциального клиента.

— убедитесь в том, что форма входа имеет поля для логина и пароля, имеет кнопку для входа и ссылку «забыли пароль?»

Формы бывают разные. Обычно это два поля: логин и пароль. Ссылка на регистрацию нужна, если ее нет отдельно на главной странице. Потому что искать ее будут именно на форме входа (если не смогут найти). Ссылку «забыли пароль?» также будут искать на форме входа. Она не должна быть отдельно, чтобы пользователь не тратил время на ее поиски. Он сталкивается с проблемой «вспомнить пароль» именно на форме входа, где пытается его подобрать.

— где находиться курсор после загрузки формы входа?

— форма входа содержит ссылку на регистрацию?

— убедитесь в том, что пользователь может войти в систему с действительным логином и паролем

— как отображаются ошибки?

— должно быть видно сообщение об ошибке если введены некорректные данные

Сообщения об ошибках могут отображаться всплывающим окном или появляться текстом рядом с полем.

— сообщение об ошибке должно быть показано

— убедитесь в том, что нельзя авторизироваться с неверными логином и паролем

— убедитесь в том, что есть предупреждение о незаполненном поле

В форме два поля и оба они являются важными, обязательными. И если одно из них пустое, тогда это должно как то выделяться: окрашивать рамку в красный цвет, выводить предупреждение, все что угодно, но только не проходить дальше.

— введите действительный логин и неправильный пароль

— введите неправильный логин и правильный пароль

— оставьте поля пустыми и нажми enter или кликни кнопку вход

— поле логин пустое, поле пароль — правильные данные

— поле логин с правильными данными, поле пароль — пустое

— введите неправильные логин и пароль

— есть ли предупреждение, что превышен лимит символов в поле логин или в поле пароль?

У каждого поля может быть ограничение на количество введенных символов. Другое дело отображается ли оно и если да, то как?

Возьмем поле «логин».

Здравый смысл подсказывает, что это ограничение должно быть одинаково как для формы входа, так и для формы регистрации. Мы можем посмотреть

в форме регистрации, какие ограничения стоят для поля «логин». Например, в форме регистрации для поля логин ограничение не больше 40 символов.

Тогда даже если в форме входа логин принимает больше 40 символов — действительным данным это никак не помешает.

Максимум неудобства, если пользователь ошибется и введет лишний символ в свой 40 символьный логин. В этом случае желательно чтоб поле «логин» из формы входа имело столько же ограничений, сколько и поле «логин» из формы регистрации. Когда в поле логин (формы входа) стоит ограничение меньше 40 символов, могут возникнуть трудности с действительными данными. (40 символов рассматриваются условно. Нет никакого правила определяющего максимальное количество символов для логина).

Если есть максимальное ограничение на количество символов в поле пароль и это никак не сообщается — могут быть проблемы. Пользователь может думать, что ввел один пароль, а на самом деле используется его укороченная версия. Например, форма может обрезать последние символы. Или просто не дать их ввести. Но пользователь об этом не догадается, так как обычно символы в поле пароль маскируются звездочками.

— убедитесь в том, что существует ограничение на общее число неудачных попыток.

Необходима защита от подбора пароля. Самый распространенный способ — ограничить число неудачных попыток. Их число может быть разным. После которого аккаунт и\или возможность входа блокируется на некоторое время.

— Есть ли чувствительность регистра у полей логина и пароля?

Чувствительность к регистру (различает ли поле маленькие буквы от больших) в поле для логина, например, быть не должно. Поле не должно считать разными логинами login и LOGIN. А вот для поля пароль это необходимо.

— поле пароль маскирует символы?(обычно звездочками)

— есть ли минимальная длина у поля пароль?

Надежность пароля можно рассматривать как длину символов (чем длиннее пароль тем тяжелее его подобрать) и количество вариантов для одного символа(чем больше вариантов, тем больше комбинации и больше времени понадобиться чтобы проверить их все). Поэтому чаще требуют чтобы пароль содержал и цифры и буквы, верхний и нижний регистр, спецсимволы — чтобы максимально расширить количество вариантов для одного символа.

```
- Арабские цифры = 10 вариантов
- Латинский алфавит = 26 вариантов
```

```
- Латинский алфавит только с нижним регистром(или только с
верхним) и цифры = 36 вариантов
- Латинский алфавит с верхним и нижним регистром = 52 варианта
- Латинский алфавит с верхним и нижним регистром + цифры
= 62  варианта
- Латинский алфавит с верхним и нижним регистром + цифры +
спецсимволы и без пробела = 94 варианта
- Латинский алфавит с верхним и нижним регистром + цифры +
спецсимволы + пробел (все символы таблицы ASCII) = 95 вариантов
- Символы расширенной таблицы ASCII - 218 вариантов
```

— логин может содержать буквы, цифры и точку.

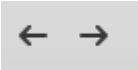
— логин не должен содержать спецсимволы

— пользователь должен осознавать, авторизирован он или нет

Система должна показывать, авторизирован пользователь или нет. Когда пользователь авторизирован, ему не должна показываться форма входа или форма регистрации. Иначе он подумает, что не авторизирован.

— нажатие кнопки логаута переводит на форму входа?

— как реагирует система на кнопки браузера «вперед» и «назад» после авторизации? А как после логаута?

Стрелками браузера  удобно пользоваться если нужно вернуться на несколько страниц назад, а потом обратно вперед. Но такой переход по страницам не должен быть переходом из одного состояния в другое. Если пользователь уже авторизирован, то возвращение на страницу назад не должно возвращать его в состояние неизвестного посетителя. Так же и при выходе из аккаунта (логаут).

— убедитесь в том, что после авторизации нажав кнопку в браузере «назад» пользователь не разлогинится обратно

— кнопка назад в браузере не должна возвращать не авторизированного пользователя обратно авторизированным

— проверьте работу чекбокса «запомнить меня»(если есть)

Если на странице входа в систему предусмотрена возможность сохранения имени пользователя и пароля, с помощью флажка, то пользователю не должно быть предложено повторно ввести учетные данные.

— копипаст разрешен для поля логина, но не желателен для поля пароль

Почему запрещается копипаст для поля пароль? Иногда бывает риск зацепить лишний пробел при копировании из файла в поле для пароля.

Чек-лист для тестирования формы входа мобильного приложения

Приложение: [Название приложения]

Версия: [Версия приложения]

Платформа: iOS / Android / Кросс-платформенное

Тестировщик: [Ваше имя/ник]

Дата: [Дата]

1. Функциональное тестирование (Functional Testing)

Цель: Проверить основную функциональность формы.

Успешный вход с валидными данными: Корректный email/логин и пароль.

Блокировка при неверных данных: Сообщения об ошибке при неверном логине/пароле.

Проверка полей ввода:

Поле "Логин/Email" принимает корректный формат (email, телефон, имя пользователя в зависимости от требований).

Поле "Пароль" скрывает вводимые символы (по умолчанию).

Кнопка "Войти" неактивна, пока оба поля не заполнены (опционально, в зависимости от дизайна).

Переключение видимости пароля: Иконка "глаз" работает (скрывает/показывает символы).

Очистка полей: Работает иконка "очистить" (X) в полях ввода (если есть).

Вход с помощью отпечатка пальца / Face ID (Biometric Login): Если поддерживается.

Вход через социальные сети / Google / Apple ID: Если предусмотрено.

Валидация в реальном времени (если есть): Подсвечивание ошибок до нажатия "Войти" (например, неверный формат email).

2. Тестирование пользовательского интерфейса (UI Testing)

Цель: Проверить соответствие макетам и удобство использования.

Верстка: Все элементы отображаются корректно на разных разрешениях и размерах экрана.

Выравнивание: Тексты, поля и кнопки правильно выровнены.

Шрифты: Используются корректные шрифты, размеры и цвета согласно гайдлайнам.

Адаптация под светлую/темную тему: Корректное отображение в обоих режимах (если поддерживается).

Интерактивность: Состояния кнопок (Normal, Pressed, Disabled) визуально различимы.

Масштабирование текста: При увеличении шрифта в настройках системы форма не "ломается", текст не обрезается.

Локализация: Все тексты переведены, форма корректно отображается для RTL-языков (например, арабский).

3. Тестирование удобства использования (UX Testing)

Цель: Проверить логику и комфорт взаимодействия.

Навигация:

Тап по полю активирует клавиатуру соответствующего типа (email, обычный текст, цифры).

Клавиша "Next" на клавиатуре переключает фокус между полями (Логин → Пароль).

Клавиша "Go" / "Enter" на клавиатуре при фокусе в поле пароля выполняет действие "Войти".

Ссылки и переходы:

Кнопка/ссылка "Забыли пароль?" ведет на соответствующий экран восстановления.

Кнопка/ссылка "Регистрация" ведет на экран регистрации.

Обратная связь:

При успешном входе есть индикатор загрузки (лоадер).

Сообщения об ошибках понятны, не содержат технических деталей (например, "Неверный логин или пароль" вместо "Ошибка 401").

После ошибки поле пароля очищается, фокус остается на нем.

Запоминание данных: Проверить работу чекбокса "Запомнить меня" (если есть). Внимание: с точки зрения безопасности на мобильных устройствах это часто реализуют через безопасное хранение в Keychain/Keystore.

4. Тестирование безопасности (Security Testing)

Цель: Проверить защищенность данных и сессии.

Защита от сниффинга: Трафик при авторизации должен передаваться по HTTPS.

Пароль скрыт: Ввод в поле пароль не отображается открытым текстом.

Защита от скриншотов: На экране ввода пароля (особенно в банковских приложениях) часто запрещают скриншоты. Проверить.

Брутфорс-защита: После 3-5 неудачных попыток входа должна быть временная блокировка или ввод CAPTCHA.

Безопасное хранение: При выходе из приложения или переустановке токен/сессия должны инвалидироваться (в зависимости от требований).

Отсутствие утечки данных в логах: В логи устройства/приложения не должны попадать логины, пароли, токены.

5. Тестирование производительности (Performance Testing)

Цель: Проверить скорость реакции и поведение при нагрузке.

Скорость отклика: Время между нажатием "Войти" и получением ответа (успех/ошибка) не должно превышать 2-3 секунд.

Поведение при потере связи:

При нажатии "Войти" и пропадании сети появляется информативное сообщение.

После восстановления сети можно повторить попытку без перезагрузки формы.

Работа с медленным соединением: Форма не "зависает", есть индикатор процесса, возможность отменить запрос (опционально).

6. Интеграционное и стороннее тестирование

Цель: Проверить взаимодействие с другими системами.

Работа с клавиатурами от сторонних производителей (SwiftKey, Gboard и т.д.).

Взаимодействие с менеджерами паролей и автозаполнением (Google Autofill, iCloud Keychain, LastPass).

Корректный переход после входа: Открывается нужный экран (Home, Dashboard), а не форма входа снова.

Выход из аккаунта (Logout): После выхода возврат на форму входа, повторный вход возможен.

7. Кросс-платформенное и контекстное тестирование

Цель: Проверить поведение в различных условиях устройства.

Смена ориентации: Поворот экрана не ломает форму.

Прерывания (Interruptions):

Вход во время ввода пришел звонок/SMS. После возврата в приложение форма и введенные данные сохранились.

Уведомление перекрыло часть формы. После его исчезновения форма корректна.

Переход в фоновый режим и возврат.

Работа на разных версиях ОС (минимальная и целевая, указанные в требованиях).

Задание 2.

Протестируйте форму входа некоторого мобильного или веб-приложения, подготовьте собственный чек-лист тестирования на основе стандартного, адаптировав его под тестируемое приложение. Заполните поля чек-листа.

Лабораторная работа 2

Построение баг-репорта

Структура баг-репорта.

Шапка	
Короткое описание (Summary)	Короткое описание проблемы, явно указывающее на причину и тип ошибочной ситуации.
Проект (Project)	Название тестируемого проекта
Компонент приложения (Component)	Название части или функции тестируемого продукта
Номер версии (Version)	Версия на которой была найдена ошибка
Серьезность (Severity)	Наиболее распространена пятиуровневая система градации серьезности дефекта: • S1 Блокирующий (Blocker) • S2 Критический (Critical) • S3 Значительный (Major) • S4 Незначительный (Minor) • S5 Тривиальный (Trivial)
Приоритет (Priority)	Приоритет дефекта: • P1 Высокий (High) • P2 Средний (Medium) • P3 Низкий (Low)
Статус (Status)	Статус бага. Зависит от используемой процедуры и **жизненного цикла бага** (bug workflow and life cycle)
Автор (Author)	Создатель баг репорта
Назначен на (Assigned To)	Имя сотрудника, назначенного на решение проблемы

Окружение	
ОС / Сервис Пак и т.д. / Браузера + версия / ...	Информация об окружении, на котором был найден баг: операционная система, сервис пак, для WEB тестирования - имя и версия браузера и т.д.
...	
Описание	
Шаги воспроизведения (Steps to Reproduce)	Шаги, по которым можно легко воспроизвести ситуацию, приведшую к ошибке.
Фактический Результат (Result)	Результат, полученный после прохождения шагов к воспроизведению
Ожидаемый результат (Expected Result)	Ожидаемый правильный результат
Дополнения	
Прикрепленный файл (Attachment)	Файл с логами, скриншот или любой другой документ, который может помочь прояснить причину ошибки или указать на способ решения проблемы

Градация Серьезности дефекта (Severity)

S1 Блокирующая (Blocker)

Блокирующая ошибка, приводящая приложение в нерабочее состояние, в результате которого дальнейшая работа с тестируемой системой или ее ключевыми функциями становится невозможна. Решение проблемы необходимо для дальнейшего функционирования системы.

S2 Критическая (Critical)

Критическая ошибка, неправильно работающая ключевая бизнес логика, дыра в системе безопасности, проблема, приведшая к временному падению сервера или приводящая в нерабочее состояние некоторую часть системы, без возможности решения проблемы, используя другие входные точки. Решение проблемы необходимо для дальнейшей работы с ключевыми функциями тестируемой системой.

S3 Значительная (Major)

Значительная ошибка, часть основной бизнес логики работает некорректно. Ошибка не критична или есть возможность для работы с тестируемой функцией, используя другие входные точки.

S4 Незначительная (Minor)

Незначительная ошибка, не нарушающая бизнес логику тестируемой части приложения, очевидная проблема пользовательского интерфейса.

S5 Тривиальная (Trivial)

Тривиальная ошибка, не касающаяся бизнес логики приложения, плохо воспроизводимая проблема, малозаметная посредствам пользовательского интерфейса, проблема сторонних библиотек или сервисов, проблема, не оказывающая никакого влияния на общее качество продукта.

Градация Приоритета дефекта (Priority)

P1 Высокий (High)

Ошибка должна быть исправлена как можно быстрее, т.к. ее наличие является критической для проекта.

P2 Средний (Medium)

Ошибка должна быть исправлена, ее наличие не является критичной, но требует обязательного решения.

P3 Низкий (Low)

Ошибка должна быть исправлена, ее наличие не является критичной, и не требует срочного решения.

Порядок исправления ошибок по их приоритетам:

High -> Medium -> Low

Требования к количеству открытых багов

Следующий подход к определению требований к количеству открытых багов:

- Наличие открытых дефектов P1, P2 и S1, S2, считается неприемлемым для проекта. Все подобные ситуации требуют срочного решения и идут под контроль к менеджерам проекта.
- Наличие строго ограниченного количества открытых ошибок P3 и S3, S4, S5 не является критичным для проекта и допускается в выдаваемом приложении. Количество же открытых ошибок зависит от размера проекта и установленных критериев качества.

Все требования к открытым ошибкам оговариваются и документируются на этапе принятия решения о качестве разрабатываемого продукта.

Баг репорт - это технический документ и в связи с этим хотим отметить, что язык описания проблемы должен быть техническим. Должна использоваться правильная терминология при использовании названий элементов пользовательского интерфейса (editbox, listbox, combobox, link, text area, button, menu, popup menu, title bar, system tray и т.д.), действий пользователя (click link, press the button, select menu item и т.д.) и полученных результатах (window is opened, error message is displayed, system crashed и т.д.).

Требования к обязательным полям баг репорта

Отметим, что обязательными полями баг репорта являются: **короткое описание** (*Bug Summary*), **серьезность** (*Severity*), **шаги к воспроизведению** (*Steps to reproduce*), **результат** (*Actual Result*), **ожидаемый результат** (*Expected Result*). Ниже приведены требования и примеры по заполнению этих полей.

Короткое описание

Название говорит само за себя. В одном предложении вам надо уместить смысл всего баг репорта, а именно: коротко и ясно, используя правильную терминологию сказать что и где не работает. Например:

1. Приложение зависает, при попытке сохранения текстового файла размером больше 50Мб.

2. Данные на форме "Профайл" не сохраняются после нажатия кнопки "Сохранить".

В дополнение предлагаем изучить *Принцип "Где? Что? Когда?"*:

"В чем этот принцип состоит?

Составьте предложение, в котором факты дефекта изложены в следующей последовательности:

- ***Где?**: В каком месте интерфейса пользователя или архитектуры программного продукта находится проблема. Причем, начинайте предложение с существительного, а не предлога.*
- ***Что?**: Что происходит или не происходит согласно спецификации или вашему представлению о нормальной работе программного продукта. При этом указывайте на наличие или отсутствие объекта проблемы, а не на его содержание (его указывают в описании). Если содержание проблемы варьируется, все известные варианты указываются в описании.*
- ***Когда?**: В какой момент работы программного продукта, по наступлению какого события или при каких условиях проблема проявляется.*

Почему последовательность должна быть именно такой? В таком виде незнакомые дефекты удобнее сортировать по summary как показывает практика (ведь, скорее всего, именно среди дефектов других инженеров будет производиться поиск дубликатов). Если вы другого мнения - придумайте свою последовательность, но она должна стать единой для всех без исключения членов проекта, иначе вы не добьетесь необходимого результата."

Серьезность

В двух словах можно отметить, что если проблема найдена в ключевой функциональности приложения и после ее возникновения приложение становится полностью недоступно, и дальнейшая работа с ним невозможна, то она **блокирующая**. Обычно все блокирующие проблемы находятся во время первичной проверки новой версии продукта (**Build Verification Test**, **Smoke Test**), т.к. их наличие не позволяет полноценно проводить тестирование. Если же

тестирование может быть продолжено, то серьезность данного дефекта будет **критическая**.

На счет **значительных**, **незначительных** и **тривиальных** ошибок вопрос достаточно прозрачный и на наш взгляд не требует лишних объяснений.

Шаги к воспроизведению / Результат / Ожидаемый результат

Очень важно четко описать все шаги, с упоминаем всех вводимых данных (имени пользователя, данных для заполнения формы) и промежуточных результатов.

Например:

Шаги к воспроизведению

1. Войдите в системы: Пользователь Тестер1, пароль xxxXXX

--> Вход в систему осуществлен

2. Кликните линк Профайл

--> Страница Профайл открылась

3. Введите Новое имя пользователя: Тестер2

4. Нажмите кнопку Сохранить

Результат

На экране появилась ошибка. Новое имя пользователя не было сохранено

Ожидаемый результат

Страница профайл перегрузилась. Новое значение имени пользователя сохранено.

Основные ошибки при написании багов репортов

Недостаточность предоставленных данных

Не всегда одна и таже проблема проявляется при всех вводимых значениях и под любым вошедшим в систему пользователем, поэтому настоятельно рекомендуется вносить все необходимые данные в баг репорт

Определение серьезности

Очень часто происходит либо завышение, либо занижение серьезности дефекта, что может привести к неправильной очередности при решении проблемы.

Язык описания

Часто при описании проблемы используются неправильная терминология или сложные речевые обороты, которые могут ввести в заблуждение человека, ответственного за решение проблемы.

Отсутствие ожидаемого результата

В случаях, если вы не указали, что же должно быть требуемым поведением системы, вы тратите время разработчика, на поиск данной информации, тем самым замедляете исправления дефекта. Вы должны указать пункт в требованиях, написанный тест кейс или же ваше личное мнение, если эта ситуация не была документирована.

Задание. Выполнить поиск багов и оформление баг-репорта (не менее 3 багов).

Рассмотреть сайты с целью поиска багов: https://www.orange-elephant.ru/, https://flowwow.com/krasnodar/, https://sadovod-opt.com/ и др.

Лабораторная работа 3

Оформление чек-листов. Построение классов эквивалентности. Тестирование граничных значений

Чек-лист.

Контрольный список/лист проверок – это список проверок, которые помогают тестировщику протестировать приложение или отдельные функции. Основная цель чек-листа состоит в том, чтобы вы не забыли проверить всё, что планировали. Классический чек-лист состоит из:

- 1-й столбец: заголовки тест-кейсов, структурированные по разделам/функционалу, или любые определенные составителем пункты;
- 2-й столбец для отметки: pass/fail;
- 3-й столбец опционально под заметки.

Если чек-лист используется еще и для наглядного отображения хода тестирования (а-ля test run), 2-й столбец может иметь опции: пусто (еще не

проверялось)/успех/ошибка/пропущено или заблокировано (например, другим дефектом).

Задание 1.

Веб-форма оформления заказа

Контекст:

Вы тестировщик в интернет-магазине. Вам нужно протестировать ключевую бизнес-логику – финальный шаг оформления заказа.

Форма содержит:

Поле "Email" (обязательное, валидация формата).

Поле "Номер телефона" (обязательное, маска: +7 (XXX) XXX-XX-XX).

Радиокнопки "Способ доставки": "Курьер" (бесплатно от 3000 руб.) и "Самовывоз" (всегда бесплатно).

Поле "Промокод" (необязательное). Применение промокода дает скидку 10%.

Блок "Итого": Динамически отображает: Сумма товаров, Доставка, Скидка, К оплате.

Кнопка "Оплатить".

Исходные данные: В корзине 1 товар за 2500 рублей.

Задание:

Протестируйте взаимосвязи и логику формы. Особое внимание:

- Логика расчета доставки.
- Применение/отмена промокода.
- Корректность итогового расчета.

Составьте чек-лист, сфокусированный на функциональности и бизнес-логике. Включите сценарии, которые проверяют, что деньги не будут потеряны для компании (например, двойная скидка, бесплатная доставка при невыполнении условия).

Построение классов эквивалентности.

Рассмотрим ипотечную компанию "Гуфи" (ИКГ). Она будет выдавать ипотеки людям с доходами от $1000 до $83333 в месяц. Те, у кого доходы ниже $1000 в месяц, не имеют на это права. Тем, у кого доходы выше $83333 в месяц, не нужна ипотека, т.к. они просто рассчитываются наличными.

Для корректного входного значения можно было бы выбрать $1342 в месяц. Для некорректных - $123 в месяц и $90000 в месяц.

1) Непрерывные классы эквивалентности.

Если входное состояние принимает дискретные значения в пределах диапазона допустимых значений, то обычно существует один корректный и два некорректных класса. ИКГ выдаст ипотеку для покупки от одного до пяти домов. Ноль или меньше домов, а также шесть и больше – не корректные входные значения. Также это не могут быть ни дробные, ни десятичные значения, такие как 2 1/2 или 3,14159.

2) Дискретные классы эквивалентности

Для корректного входного значения можно выбрать два дома. Некорректными могут быть -2 и 8. ИКГ может выдавать ипотеки только человеку. Они не выдают ипотеки компаниям, трастам, партнерствам или другим легальным организациям.

3) Классы эквивалентности с одним возможным значением.

Для верных входных данных будем использовать "человек". Для неверных можно использовать "корпорация" или "траст" или любую другую случайную строку. Следует создать как минимум один тестовый сценарий с неверными данными.

ИКГ выдают ипотеки под квартиры, таунхаусы и частные дома. Они не выдают ипотеки под дома на двух хозяев, передвижные дома, домики на дереве или другие типы жилья.

4) Классы эквивалентности с несколькими возможными значениями.

Для верных входных данных можно выбрать значение "квартира", "таунхаус" или "частный дом". Несмотря на то, что правило указывает выбрать один тест из класса эквивалентности, более комплексный подход заключается в создании тестового сценария на каждое значение в классе эквивалентности. Это имеет смысл, если список верных значений мал. Но, если у нас список из пятидесяти городов, то все зависит от рисков организации, если тестировщик пропустит что-либо жизненно важное.

Создавать отдельные тесты для каждого отдельного класса эквивалентности всех входных данных, вводимых в систему неэффективно. Удобнее создавать тестовые сценарии, которые будут проверять некоторое число полей ввода одновременно. Например, мы можем создать один тестовый сценарий для следующих комбинаций входных данных:

Таблица 1. Тестовый сценарий для верных входных данных.

Ежемесячный доход	Количество жилых помещений	Заявитель	Вид жилья	Результат
$5000	2	Человек	Квартира	корректное значение

Но использовать такой подход для неверных данных опасно.

Таблица 2. Тестовый сценарий неверных данных. Это плохой подход.

Ежемесячный доход	Количество жилых помещений	Заявитель	Вид жилья	Результат
$100	8	Партнерство	Домик на дереве	некорректное значение

Если система принимает эти данные как верные, то ясно, что система не проверяет все четыре поля для ввода как следует. Но если система отклонит эти данные как неверные, то тестировщик не сможет понять, какое поле было отклонено.

Задание 2.

Тестирование калькулятора стоимости доставки

Контекст

Вы работаете в компании "Быстрая Доставка". Вам нужно протестировать модуль расчета стоимости доставки в новом личном кабинете клиента. Логика расчета основана на весе посылки.

Бизнес-правила

Система расчета стоимости работает по следующим тарифам:

1. **Лёгкие посылки**: 0 - 5 кг → 300 рублей (фиксированная цена)

2. **Стандартные посылки**: 5.01 кг - 20 кг → 300 рублей + 50 рублей за каждый полный килограмм сверх 5 кг

Пример: 7.3 кг = 5 кг + 2.3 кг. Берутся полные килограммы сверх: 2 кг. Стоимость = 300 + (2 * 50) = 400 руб.

3. **Тяжёлые посылки**: 20.01 кг - 50 кг → Фиксированная ставка 1050 рублей

4. **Посылки не принимаются**: Вес ≤ 0 кг или > 50 кг → показывается сообщение: "Вес должен быть в диапазоне от 0.01 до 50 кг"

Поле ввода "Вес" (в кг):

- Тип: Числовое поле с плавающей точкой.
- Допустимые символы: цифры (0-9) и точка (.) в качестве разделителя.
- Максимальная точность: 2 знака после запятой.
- Интерфейс: Поле + кнопка "Рассчитать".

Задание

Часть 1: Анализ и проектирование тестов

Используя технику *классов эквивалентности* и *граничных значений*, выполните следующие шаги:

1. Определите классы эквивалентности (ЭК) для входных данных (веса):
   - Валидные классы (по бизнес-правилам).
   - Невалидные классы.
2. Для каждого валидного и невалидного класса определите граничные значения:
   - Укажите нижнюю и верхнюю границы для каждого интервала.
   - Не забудьте про значения внутри интервала и за его пределами.
3. Составьте сводную таблицу тестовых сценариев, которая будет включать:
   - Номер теста.
   - Категория (валидный/невалидный, ЭК).
   - Входное значение (вес в кг).
   - Ожидаемый результат (стоимость или сообщение об ошибке).
   - Обоснование (какую границу/класс проверяем).

Пример строки:

№	Категория	Вход (кг)	Ожидаемый результат	Обоснование (ГЗ/ЭК)
1	Валидный ЭК1 (0-5 кг), нижняя граница	0.01	300 руб.	Минимальный допустимый вес

Часть 2: Создание чек-листа

На основе вашего анализа из Части 1 создайте детализированный чек-лист для тестирования поля "Вес" и функции расчета.

Чек-лист должен включать разделы:

1. Функциональное тестирование (позитивные сценарии): Проверка расчета по всем тарифным зонам с использованием подобранных вами значений из таблицы.
2. Тестирование валидации и обработки ошибок (негативные сценарии): Проверка невалидных входных данных (включая граничные).

3. Тестирование поля ввода (UI/UX): Проверка формата ввода, точности, поведения интерфейса.

4. Дополнительные нефункциональные проверки (минимум 3 пункта на ваше усмотрение).

Лабораторная работа 4

Тестирование таблиц решений. Попарное тестирование

Таблицы решений представляют собой комплекс бизнес-правил, основанных на заданных условиях.

Таблица 1. Основной формат таблицы решений

	Правило 1	**Правило 2**	**...**	**Правило p**
Условия				
Условие-1				
Условие-2				
...				
Условие-m				
Действия				
Действие-1				
Действие-2				
...				
Действие-n				

Пример. Компания по автострахованию дает скидку водителям, которые состоят в браке и/или хорошо учатся. Давайте начнем с **условий**. Следующая таблица решений имеет два условия, каждое из которых может принимать значение "Да" или "Нет".

Таблица 2. Таблица решений с двумя бинарными условиями

	Правило 1	Правило 2	Правило 3	Правило 4
Условия				
Состоит в браке?	Да	Да	Нет	Нет
Хороший студент?	Да	Нет	Да	Нет

Каждое правило является причиной "запуска" действия. Каждое правило может задать действие, уникальное для этого правила, или правила могут иметь общие действия.

Таблица 3. Добавление единственного действия в таблицу решений

	Правило 1	Правило 2	Правило 3	Правило 4
Условия				
Состоит в браке?	Да	Да	Нет	Нет
Хороший студент?	Да	Нет	Да	Нет
Действия				
Скидка ($)	60	25	50	0

Для каждого правила с помощью таблицы решений можно указать более одного действия. Эти правила могут быть уникальными или быть общими.

Таблица 4. Таблица решений с несколькими действиями.

	Правило 1	Правило 2	Правило 3	Правило 4
Условия				
Условие-1	Да	Да	Нет	Нет
Условие-2	Да	Нет	Да	Нет
Действия				
Действие-1	Выполнить X	Выполнить Y	Выполнить X	Выполнить Z
Действие-2	Выполнить A	Выполнить B	Выполнить B	Выполнить B

В такой ситуации выбрать тесты просто: каждое правило (вертикальная колонка) становится тест-кейсом. Условия указывают на входные значения, а действия - на ожидаемые результаты. В предыдущих примерах используются простые бинарные условия, но условия могут быть более сложными.

Таблица 5. Таблица решений со сложными условиями.

	Правило 1	Правило 2	Правило 3	Правило 4
Условия				
Условие-1	0-1	1-10	10-100	100-1000
Условие-2	<5	5	6 или 7	>7
Действия				
Действие-1	Выполнить X	Выполнить Y	Выполнить X	Выполнить Z
Действие-2	Выполнить A	Выполнить B	Выполнить B	Выполнить B

В этой ситуации выбор тестов становится чуть более сложным - каждое правило (вертикальная колонка) становится тест-кейсом, но при этом должны быть выбраны значения, удовлетворяющие условиям.

Выбирая необходимые значения, мы создаем следующие тест-кейсы.

Таблица 6. Примерные тест-кейсы.

ID тест-кейса	Условие 1	Условие 2	Ожидаемый результат
Тест-кейс 1	0	3	Выполнить X / Выполнить A
Тест-кейс 2	5	5	Выполнить Y / Выполнить B
Тест-кейс 3	50	7	Выполнить X / Выполнить B
Тест-кейс 4	500	10	Выполнить Z / Выполнить B

Если тестируемая система имеет сложные бизнес-правила, а у бизнес-аналитиков или проектировщиков нет документации этих правил, то тестировщикам следует собрать эту информацию и представить её в виде таблицы решений. Причина проста: представляя поведение системы в такой полной и компактной форме, тест-кейсы могут быть созданы непосредственно из таблицы решений.

При тестировании для каждого правила создаётся как минимум один тест-кейс. Если состояния этого правила бинарные, то должно быть достаточно одного теста для каждого сочетания. С другой стороны, если состояние является диапазоном значений, то тестирование должно учитывать и нижнюю, и высшую границы диапазона. Таким образом, мы объединяем идею тестирования граничных значений с тестированием таблиц решений.

Для каждого правила создаётся как минимум один тест-кейс.

Чтобы создать тестовую таблицу, просто измените заголовки строк и столбцов.

Таблица 7. Таблица принятия решений преобразуется в таблицу тест-кейсов.

	Тест-кейс 1	Тест-кейс 2	Тест-кейс 3	Тест-кейс 4
Входные значения				
Условие-1	Да	Да	Нет	Нет
Условие-2	Да	Нет	Да	Нет
Ожидаемые результаты				
Действие-1	Выполнить X	Выполнить Y	Выполнить X	Выполнить Z
Действие-2	Выполнить A	Выполнить B	Выполнить B	Выполнить B

Задание 1.

Система одобрения заявки на кредитную карту

Контекст: Вы работаете в банке. Нужно протестировать бизнес-логику автоматического принятия решения по онлайн-заявке на кредитную карту "Стартовая".

Бизнес-правила

Решение принимается на основе трёх входных условий:

Возраст заявителя (A):

- A1 = От 21 до 65 лет (включительно)
- A2 = Младше 21 или старше 65

Кредитная история (B):

- B1 = Хорошая (нет просрочек за последние 5 лет)
- B2 = Плохая (есть просрочки)

Требуемый кредитный лимит (C):

- C1 = До 100 000 руб. включительно
- C2 = Свыше 100 000 руб.

Логика принятия решения (Действия):

- Одобрить (O): Карта одобряется с запрошенным лимитом.
- Одобрить с понижением лимита (OP): Карта одобряется, но лимит снижается до 100 000 руб.
- Отклонить (OT): В заявке отказывают.

– Отправить на ручную проверку (Р): Автоматическая система не может принять решение, заявку передают менеджеру.

Конкретные правила (их нужно формализовать в таблице):

1. Карта одобряется, если: Хорошая кредитная история И возраст от 21 до 65 И лимит до 100к.
2. Карта одобряется с понижением лимита, если: Хорошая кредитная история И возраст от 21 до 65, НО запрошен лимит свыше 100к.
3. Заявка отклоняется, если: Плохая кредитная история И возраст НЕ от 21 до 65 (независимо от лимита).
4. Во всех остальных случаях заявка отправляется на ручную проверку.

Задание

1. Постройте полную таблицу решений (Decision Table), отобразив все возможные комбинации условий (A, B, C) и соответствующие им действия (О, ОР, ОТ, Р).
   - Условия (Conditions) – это строки.
   - Столбцы (Rules) – это комбинации значений условий.
   - Действия (Actions) – это строки с результатами.
2. Проведите оптимизацию (свертку) таблицы. Найдите правила (столбцы) с одинаковыми действиями, которые можно объединить с помощью символа "-" (неважное условие). Создайте оптимизированную таблицу.
3. На основе оптимизированной таблицы составьте тест-кейсы. Каждый столбец (правило) = один тест-кейс. Для каждого кейса укажите:
   1. ID теста.
   2. Входные данные (возраст, кредитная история, лимит).
   3. Ожидаемый результат.
   4. Номер проверяемого бизнес-правила (из списка выше).

Попарное тестирование

Программу построения матрицы попарных тестов можно скачать:

http://www.satisfice.com

(download allpair)

В Excel следует создать таблицу с вариантами и сохранить с расширением txt (input.txt)

Browser	Client OS	Plug-in	Server	Server OS
IE 5.0	Win 95	None	IIS	Win NT
IE 5.5	Win 98	Real Player	Apache	Win 2000
IE 6.0	Win ME	Media Player	WebLogic	Linux
Netscape 6.0	Win NT			
Netscape 6.1	Win 2000			
Netscape 7.0	Win XP			
Mozilla 1.1				
Opera 7				

Команда запуска:

allpairs input.txt> output.txt

Задание 2.

Настройка уведомлений в мобильном приложении "Фитнес-Трекер"

Контекст: Нужно протестировать экран настройки, где пользователь может создать правило для уведомлений. Для тестирования выбраны 4 параметра с несколькими значениями каждый.

Параметры и их значения:

1. Тип активности (A): Бег, Велосипед, Ходьба
2. Условие срабатывания (B): Дистанция > 5 км, Время > 30 мин, Сожжено > 300 кал
3. Тип уведомления (C): Push, SMS, Email
4. Повтор (D): Однократно, Каждый день, Только в выходные

Если использовать полный перебор, получится: 3 * 3 * 3 * 3 = 81 уникальная комбинация. Тестировать все слишком долго.

Задание

1. С помощью техники попарного тестирования сгенерируйте минимальный набор тестовых комбинаций, в котором каждое значение каждого параметра хотя бы раз встречается с каждым значением любого другого параметра.

2. Создайте чек-лист для тестирования на основе сгенерированного набора. Чек-лист должен включать:

- Раздел "Конфигурации уведомлений" (ваши попарные комбинации).
- Раздел "Общие проверки" (например: "Созданное правило сохраняется в списке", "При срабатывании условия приходит выбранный тип уведомления").
- Раздел "Пограничные и негативные сценарии" (например: "Создание правила без выбора типа активности", "Выбор всех типов уведомлений одновременно, если это возможно").

3. Ответьте на вопрос:

Сколько тест-кейсов у вас получилось в попарном наборе? (Для 4 параметров 3x3x3x3 оптимальное решение обычно ~9-12 комбинаций).

Какие, на ваш взгляд, самые критичные комбинации параметров, которые обязательно должны быть протестированы, даже если они не попали в попарный набор?

Пример ожидаемого формата результата (фрагмент):

1. Попарный набор комбинаций (*первые 3 строки*):

№	Тип активности (A)	Условие (B)	Тип уведомления (C)	Повтор (D)
1	Бег	Дистанция > 5 км	Push	Однократно
2	Бег	Время > 30 мин	SMS	Каждый день
3	Бег	Сожжено > 300 кал	Email	Только в выходные

Лабораторная работа 5

Комбинаторное тестирование. Диаграмма состояний и переходов

Задание 1

Комбинаторное тестирование с ограничениями

Контекст: Конфигуратор ноутбука. Пользователь выбирает:

1. Процессор: Intel i5, Intel i7, AMD Ryzen 7
2. Оперативная память: 8 ГБ, 16 ГБ, 32 ГБ
3. Накопитель: SSD 256 ГБ, SSD 512 ГБ, SSD 1 ТБ, HDD 1 ТБ
4. Видеокарта: Интегрированная, NVIDIA GTX 1650, NVIDIA RTX 3060

Ограничения бизнес-логики:

- С Intel i5 нельзя выбрать 32 ГБ ОЗУ
- С интегрированной видеокартой нельзя выбрать NVIDIA RTX 3060
- HDD 1 ТБ доступен только с Intel i5 или AMD Ryzen 7

Задание:

1. Рассчитайте количество комбинаций при полном переборе.
2. Примените технику попарного тестирования, учитывая ограничения.
3. Сгенерируйте оптимальный тестовый набор (можно использовать инструмент или ручной подбор).
4. Объясните, какие комбинации вы исключили и почему.

Задание 2

Тестирование состояний и переходов (State Transition)

Контекст: Умная кофемашина имеет 4 состояния:

1. Выключена (Off)
2. Готова (Ready) — включена, вода есть, зерна есть
3. Приготовление (Brewing) — готовит кофе
4. Ошибка (Error) — нет воды или нет зерен

Переходы:

- Выкл → Готов: Нажата кнопка POWER при наличии воды и зерен
- Готов → Приготовление: Нажата кнопка START

- Приготовление → Готов: Прошло 2 минуты
- Готов → Ошибка: Закончилась вода ИЛИ закончились зерна
- Ошибка → Готов: Долита вода И добавлены зерна

- Любое → Выкл: Долгое нажатие POWER (3 сек)

Задание:

1. Постройте диаграмму переходов состояний.
2. Создайте таблицу переходов со всеми возможными сценариями.
3. Выделите 5 наиболее критичных тестовых сценариев и объясните, почему они критичны.
4. Придумайте 2 "зловредных" (mischievous) теста, которые проверяют неочевидное поведение.

Лабораторная работа 6

Сценарное и исследовательское тестирование

Сценарное тестирование предполагает разделение задачи на этапы (подготовка, выполнение, завершение и т.д.) и последовательное выполнение этих этапов.

Особенности этого подхода:

- четкое понимание того, какие функции покрыты тестами;
- уверенность, что все задокументированные тест-кейсы будут пройдены в срок;
- быстрое и легкое подключение нового специалиста на проект благодаря наличию подробных сценариев.

Пример. Тестируется почта, подробное сценарное тестирование может выглядеть так:

- *Перейдите на стартовую страницу*
- *Нажмите на кнопку «Новый»*
- *В поле «Кому» напишите j.cannegieter@squerist.nl*
- *Нажмите на кнопку «Файл»*

- *Выберите документ «Тест1»*
- *В поле «Тема» напишите «Тестовое вложение <дата>»*
- *Откройте почтовый ящик j.cannegieter*
- *Проверьте, доставлено ли сообщение и вложение.*

Для этого же примера сценарий общего тестирования может выглядеть так:

- *Создайте и отправьте письмо с вложением одному получателю*
- *Проверьте, доставлено ли сообщение и вложение*
- *https://habr.com/ru/companies/jugru/articles/341852/*

Задание. Сценарное тестирование.

Откройте сайт интернет-магазина. Необходимо описать сценарий тестирования (выбрать три задачи из списка ниже). Оформить отчет.

1. Ориентирование в категориях каталога. Вариант 1
2. Ориентирование в категориях каталога. Вариант 2
3. Фильтрация по заданным параметрам
4. Сброс фильтров
5. Сортировка по заданному параметру
6. Изучение карточки товара в каталоге
7. Изучение страницы с подробным описанием товара
8. Поиск товара по заданным параметрам
9. Сравнение товаров
10. Ориентирование в подборках товаров
11. Добавление товара в избранное
12. Работа с изображением товара
13. Ориентирование в стоимости товара

14. Взаимодействие с чужими отзывами

15. Считываемость доступного кол-ва товара

16. Обращение в поддержку

17. Заказ в один клик

18. Добавление товаров в корзину

19. Удаление товара из корзины

20. Очистка корзины

21. Применение промокодов

22. Поиск ближайшей точки по карте

23. Понимание условий возврата товаров

24. Понимание условий программы лояльности

25. Понимание Push/SMS о статусе заказа

26. Считываемость баннеров

27. Перелистывание карусели

28. Отмена заказа

29. Треккинг заказа

30. Ориентирование в стоимости опций и заказа

31. Считываемость рекомендуемых товаров

32. Поиск заданной точки по карте

33. Оформление доставки

34. Смена города/региона

35. Работа с кликабельными тэгами

36. Оставление фидбека о товаре

37. Взаимодействие с видео о товаре

38. Передача ссылок и шеринг информации о товаре

39. Использование списка избранного

40. Повторение прошлых заказов

41. Редактирование профиля

42. Редактирование состава заказа в корзине

43. Изменений опций заказа

44. Выбор способа оплаты

45. Выбор способа получения товара

46. Считываемость экранов/сообщений об ошибке

47. Взаимодействие с всплывающими подсказками

48. Регистрация

49. Поиск товаров, похожих на просматриваемый

50. Считываемость продвигаемых товаров

51. Оформление самовывоза

52. Свободный поиск товара

53. Работа с поисковой строкой

54. Оплата заказа

55. Поиск товара через каталог

Лабораторная работа 7

Отчеты о тестировании

Отчет о тестировании — документ, включающий в себя результаты работ по тестированию и содержащий информацию, достаточную для соотнесения текущей ситуации с тест-планом и принятия необходимых управленческих решений.

Основные принципы составления отчета о тестировании:

- **Подробность**: необходимо подробно описывать проделанную работу. Не стоит использовать абстрактные слова, формы и выражения.
- **Технический стиль:** используйте технический стиль в отчете.
- **Стандарт:** используйте общепринятый на проекте шаблон отчета.

Типы отчетов

Выделяют два типа отчетов — высокоуровневые и низкоуровневые. Высокоуровневые отчеты отличаются тем, что предоставляют общую оценку о качестве продукта, а низкоуровневые — содержат более детальную информацию о тестировании.

К задачам **высокоуровневых отчетов** относятся:

- сбор, агрегация и предоставление в удобной для восприятия форме объективной информации о результатах работы;
- оценка текущего статуса тестирования и прогресс;
- подсвечивание существующих проблем;
- прогноз развития ситуации и фиксация рекомендаций по устранению дефектов.

К задачам **низкоуровневых отчетов** относятся:

- оценка объема и качества выполненных работ;
- информирование о текущих проблемах;
- сравнение текущего прогресса с планом;
- полная информация о текущем состоянии качества проекта.

Содержание отчета о тестировании

Хоть в каждой компании свои требования, свой формат, свои цели, типичный шаблон сводного отчета о тестировании обычно содержит:

Информацию о проекте — раздел, в котором кратко описывается содержание отчета. Например, название проекта, фичи, тестируемая версия и т.д.

Команда тестировщиков — список специалистов, задействованных в тестировании проекта, с указанием закрепленного за каждым участником тестируемого функционала.

Описание процесса тестирования — в этом разделе кратко описывается то, как происходил процесс тестирования, а именно какие использовались методы, техники, инструменты.

Краткое описание — в данном разделе дается краткое описание того, что было протестировано (версия, номер сборки), что тормозит процесс тестирования (критичные дефекты, недоработанная функциональность, пробелы в требованиях).

Расписание — указываются временные сроки тестирования функционала.

Статистика по дефектам — количество дефектов, обнаруженных за отчетный период, с указанием степени важности, срочности, места обнаружения и прочего.

Список новых дефектов — список дефектов, которые обнаружились в отчётном периоде, с указанием их основных атрибутов.

Список устраненных дефектов — список дефектов, которые были устранены командой разработки в отчетном периоде.

Качество объекта тестирования — экспертное заключение об уровне качества продукта.

Рекомендации — выводы по результатам тестирования и рекомендации по повышению его эффективности в будущем.

Приложения — графики, таблицы, диаграммы, демонстрирующие числовые характеристики процесса тестирования.

Пример отчета о тестировании сайта

«

Optimizing the performance of computer systems has always been an art relegated to a few individuals who happen to have the «right skills»

Amir H. Majidimehr

ОТЧЕТ ПО РЕЗУЛЬТАТАМ ТЕСТИРОВАНИЯ САЙТА

https://www.apple.com/ru/

методом 50 Функциональных и Регрессионных тест-кейсов

для тестирования web-приложений

PerformanceLab
Тестирование и обеспечение качества

Оглавление

Краткое описание 3
Сведения о ходе испытаний 4
Выводы по результатам испытаний 9
Приложение 10

PerformanceLab
Тестирование и обеспечение качества

Краткое описание

В ходе тестирования был применен метод 50 тест - кейсов, который проверяет работу сайта apple.com/ru/

Суть тестирования заключается в проверке работы всех обязательных функций сайта и его корректное отображение во всех актуальных браузерах.

Сведения о ходе испытаний

Таблица 1. Тест-кейсы, пройденные в ходе тестирования сайта https://www.apple.com/ru/

№ Тест-кейса	Название проверки	Результат	Замечания и рекомендации
	Кроссбраузерное тестирование		
1	Приложение корректно отображается и функционирует во всех актуальных браузерах (IE, Edge, Chrome, Firefox, Safari,...).	Отрицательно	Отсутствует эффект скроллинга в браузере IE
2	Приложение корректно отображается и функционирует во всех актуальных версиях браузеров.	Отрицательно	Отсутствует эффект скроллинга в браузере IE
3	Корректная обработка тайм-аутов	Положительно	
4	Работоспособность при удалении cookies во время использования приложения.	Положительно	
5	Работоспособность при удалении cookies после использования приложения.	Положительно	
	Работа с формами		
6	Обязательные поля должны быть заполнены до подтверждения ввода данных.	Положительно	
7	Обязательные поля должны быть явно обозначены.	Положительно	
8	Если пользователь нажал кнопку подтверждения ввода данных с незаполненными обязательными полями, эти поля должны быть подсвечены и отображено сообщение об ошибке.	Положительно	

№ Тест-кейса	Название проверки	Результат	Замечания и рекомендации
9	Сообщение об ошибке не отображается, если не заполнены необязательные поля.	Положительно	
10	Текстовые поля допускают ввод специальных символов.	Положительно	
11	При вводе количества знаков меньше минимального отображается сообщение об ошибке.	Положительно	
12	При вводе количества знаков больше максимального отображается сообщение об ошибке.	Положительно	
13	Если обязательное поле заполнено не полностью, отображается сообщение об ошибке.	Положительно	
14	Сообщение об ошибке отображается при попытке ввести недопустимые символы (например, ввод текста в поле, принимающее только числа).	Положительно	
	Авторизация и работа с профилем		
15	Создайте новый аккаунт.	Положительно	
16	Авторизуйтесь с существующим аккаунтом.	Положительно	
17	Восстановите/сбросьте забытый пароль.	Положительно	
18	При вводе неверного пароля отображается сообщение об ошибке и ссылка на страницу восстановления пароля.	Положительно	
19	Отредактируйте профиль пользователя.	Положительно	

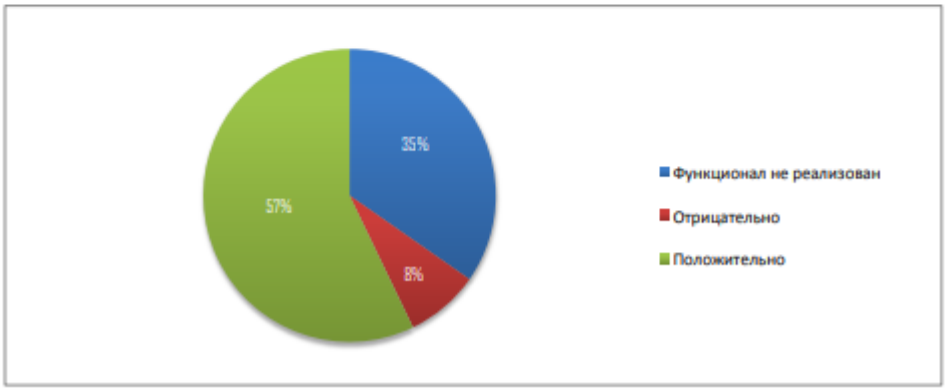
№ Тест-кейса	Название проверки	Результат	Замечания и рекомендации
20	Зарегистрируйте нового пользователя, используя аккаунт социальных сетей.	Функционал не реализован	
21	Авторизуйтесь, используя аккаунт социальных сетей.	Функционал не реализован	
22	Загрузите в профиль аватар/картинку.	Функционал не реализован	
23	Отредактируйте аватар/картинку.	Функционал не реализован	
24	Удалите аватар/картинку.	Функционал не реализован	
25	При регистрации нового пользователя ему приходит подтверждение по электронной почте.	Положительно	
26	При оформлении подписки на уведомления пользователю приходит подтверждение по электронной почте.	Отрицательно	
27	При смене пароля пользователю приходит подтверждение по электронной почте	Положительно	
	Навигация		
28	При выполнении действий (например, выход из аккаунта) и манипуляциях с данными (например, удаление файла) отображается сообщение с подтверждением.	Отрицательно	

№ Тест-кейса	Название проверки	Результат	Замечания и рекомендации
29	Корректность работы сортировки.	Функционал не реализован	
30	Корректность работы фильтрации.	Положительно	
31	Корректность работы кнопок и ссылок в меню навигации.	Положительно	
32	Отображение страницы с ошибкой 404, если пользователь указал неверный путь к файлу или URL.	Положительно	
	Числа и даты		
33	Осуществляется проверка на корректность дат	Положительно	
34	Корректная обработка високосного года, отсутствует сообщение об ошибке	Положительно	
35	Числовые поля не допускают ввода текста, отображается сообщение об ошибке.	Положительно	
36	При попытке ввода отрицательных чисел, отображается сообщение об ошибке, если предусмотрен ввод строго положительных чисел.	Положительно	
37	При попытке деления на ноль отображается сообщение об ошибке.	Функционал не реализован	
38	Поля специальных форматов (например, процентные, денежные) отображаются корректно.	Положительно	
39	Всплывающие уведомления об ошибке и предупреждения работают корректно.	Положительно	

№ Тест-кейса	Название проверки	Результат	Замечания и рекомендации
	Интеграция с социальными сетями		
40	Корректное отображение кнопок социальных сетей.	Функционал не реализован	
41	При нажатии на кнопку "поделиться" отображается всплывающее окно, в котором доступна функция редактирования сообщения перед отправкой.	Функционал не реализован	
42	Корректное отображение окна авторизации с помощью аккаунта социальных сетей.	Функционал не реализован	
43	Корректное отображение аватара пользователя из профиля социальных сетей.	Функционал не реализован	
44	Пользователь может оставлять комментарии на вашем сайте, используя аккаунт социальных сетей.	Функционал не реализован	
	Загрузка и скачивание файлов		
45	Файлы успешно загружаются.	Функционал не реализован	
46	Слишком тяжелые файлы не загружаются, и отображается сообщение об ошибке.	Функционал не реализован	
48	Файлы ошибочного формата не загружаются, и отображается сообщение об ошибке.	Функционал не реализован	

№ Тест-кейса	Название проверки	Результат	Замечания и рекомендации
49	Файлы недопустимого формата не загружаются, и отображается сообщение об ошибке.	Функционал не реализован	
50	Файлы успешно скачиваются.	Функционал не реализован	

Рисунок 1. Диаграмма результатов выполнения тест - кейсов.



PerformanceLab
Тестирование и обеспечение качества

Выводы по результатам испытаний

В данном разделе представлены выводы по результатам испытаний, а также общая статистика по дефектам, которые были выявлены в результате тестирования сайта https://www.apple.com/ru/. В период испытаний был выявлено 4 дефекта. Список дефектов приведен в Приложении.

Приложение

Таблица 2. Список дефектов

Порядковый №	№ Кейса	Краткое описание	Состояние	Критичность
1	1	Отсутствует эффект скроллинга в браузере IE	Обнаружена	Низкая
2	2	Отсутствует эффект скроллинга в браузере IE	Обнаружена	Низкая
3	26	Письмо с подтверждением не приходит	Обнаружена	Высокая
4	28	Пользователь разлогинивается без подтверждения действия	Обнаружена	Низкая

Задание

Полный цикл тестирования и отчет по модулю "Онлайн-запись к врачу"

Контекст проекта: Вы — ведущий тестировщик в компании "HealthTech Solutions". Команда разработала новый модуль "Онлайн-запись к врачу" для мобильного приложения клиник. Модуль позволяет пациентам:

1. Выбрать врача из списка.
2. Выбрать доступную дату и время из расписания.
3. Подтвердить запись, введя номер телефона для подтверждения.
4. Получить SMS с кодом подтверждения.

Текущий статус: Разработка модуля завершена. Вы провели плановое тестирование новой функциональности в период с 15.04.2024 по 19.04.2024.

Исходные данные для отчета

1. Объем тестирования:

Протестировано: 3 основных пользовательских сценария, 42 тест-кейса.

Среда тестирования: iOS 16.4, Android 13, сервер Staging.

2. Результаты выполнения тестов:

Успешно (Passed): 35 тест-кейсов

Провалено (Failed): 7 тест-кейсов → Выявлено 5 дефектов (один дефект может "ломать" несколько кейсов).

3. Список выявленных дефектов (для включения в отчет):

ID дефекта	Краткое описание	Серьезность	Приоритет	Статус
HEALTH-101	При выборе даты "29.02.2025" (невисокосный год) календарь крашится.	Critical	High	Open
HEALTH-102	После успешной записи не приходит SMS с кодом подтверждения.	Major	High	In Progress
HEALTH-103	Кнопка "Подтвердить запись" неактивна, если в поле телефона 11 цифр (формат +7).	Major	High	Open
HEALTH-104	В списке врачей не отображается их специализация при фильтрации "Терапевт".	Minor	Medium	Open
HEALTH-105	На экране подтверждения опечатка: "Врач перезвнит вам" (пропущена "о").	Trivial	Low	Open

4. Ключевые метрики:

- Процент успешных тестов: 83% (35/42)
- Плотность дефектов: 0.12 дефекта на тест-кейс (5/42)
- Выполнено по плану: 100% запланированных тест-кейсов были выполнены.

Задание

Ваша задача — составить формальный отчет о тестировании (Test Summary Report), который будет представлен руководителю проекта, продукт-менеджеру и команде разработки.

Отчет должен содержать следующие разделы:

1. Идентификация документа (Header):

− Название проекта и модуля.

− Версия ПО.

− Дата и период тестирования.

− Автор отчета.

2. Резюме для руководства (Executive Summary):

2-3 предложения о том, что тестировалось и каков общий вывод. Можно ли выпускать модуль? Если нет, то почему.

3. Объем тестирования (Scope of Testing):

− Что включено в тестирование (функционал, среды).

− Что исключено из тестирования (например, интеграция с платежной системой, т.к. ее еще нет).

4. Методология тестирования (Testing Approach):

− Какие виды тестирования применялись (Функциональное, UI, Кросс-платформенное).

− Какие техники использовались (Эквивалентное разделение, Таблица решений — можно добавить, что тестировали логику доступности времени).

5. Результаты и метрики (Results and Metrics):

− Диаграмма/таблица с итогами выполнения тестов (Passed/Failed/Blocked).

− Сводная таблица по дефектам (как выше, но сгруппированная по серьезности).

− Ключевые метрики (процент успешных тестов, плотность дефектов).

6. Оценка качества (Quality Assessment):

− Сильные стороны модуля (что работает стабильно и хорошо).

− Критические риски (дефекты, блокирующие выпуск). Дайте оценку: Модуль готов к выпуску в Production? Аргументируйте.

7. Выводы и рекомендации (Conclusions and Recommendations):

− Основные выводы.

− Конкретные рекомендации команде:

− Что нужно исправить в первую очередь (HEALTH-101, HEALTH-102).

− Что можно исправить в следующем спринте.

− Нужно ли проводить повторное тестирование (регресс) после исправлений.

8. Приложения (Appendices):

− Ссылка на тестовую документацию (чек-листы, тест-кейсы в TestRail).

− Ссылка на отчеты о дефектах в Jira/YouTrack.

− Логи тестирования (если есть ключевые скриншоты или видео падений).

Лабораторная работа 8

Тестирование форм (полей ввода)

Используйте классы эквивалентности, граничные значения для тестирования форм. Для тестирования форм есть плагин для Chrome и Firefox — **BugMagnet**. После установки кликните правой кнопкой мыши по любому полю формы и появится возможность выбора значений для заполнения из огромного списка данных.

Таблица верификации полей

Тип данных	Корректный ввод	Некорректный ввод
Положительные целые числа	— только числа — максимально возможное значение (N) — числа внутри промежутка (N + 1) / 2	— число больше максимально возможного (N + 1) — дробные числа — отрицательные числа — строковые значения — числа + строковые значения — числа + специальные символы — Unicode (например U+0000, U+0001)
Строки	— только символы — только числа — только специальные символы — числа + символы — числа + спецсимволы — символы + спецсимволы	—
Даты	— проверить, что при выборе появляется datepicker — проверить, что поле нельзя изменить с клавиатуры — проверить, что значение поля можно скопировать, но вставить нельзя — проверить, что при выборе значения в datepicker, оно появляется в поле — проверить, что в феврале 29 дней в високосных годах — проверить, что в феврале 28 дней не в високосных годах	—

Чек-лист для веб-форм

Ниже приведены основные элементы чек-листа.

1. Расположение формы на экране до заполнения, после неправильного заполнения, после правильного заполнения.

2. Сохранение/изменение URL при отправке данных из формы.

3. Отправка формы с пустыми обязательными полями.

4. Отправка формы с некорректно заполненными полями (числа вне допустимого диапазона, длина строк превышает допустимую и т.д.)

5. Подача формы с «нелогичными данными» (например, дата рождения в будущем).

6. Отправка формы с полями, содержащими спецсимволы.

7. Восстановление значений полей после отправки формы с некорректно заполненными полями.

8. Информативные сообщения об ошибках.

9. Функциональная группировка полей формы. Информативные этикетки, подсказки.

10. Эффективность и удобство использования формы с отключенным JavaScript.

11. + использовать контрольные списки для каждого стандартного поля (текстовое, числовое, дата и т. д.)

Пример чек-листа для тестирования веб-форм.

Extra High

□ **Сохранение формы.**

□ Форма сохраняется в веб-формах (админ-панели) или SQL-таблицах.

□ **Изменение адреса отправки.**

□ E-mail, на который приходят данные из веб-формы, можно менять в административной панели.

High

□ **Актуальность адреса отправки.**

□ Прописан реальный e-mail лица, отвечающего за обработку заявок.

□ **Отправка формы.**

□ Данные из заполненной формы отправляются администратору на e-mail.

□ **Отправка уведомления пользователю.**

□ Опционально. Пользователь получает уведомление на свой e-mail об успешно полученной заявке и последующих действиях, которые от него требуются.

Навигация

□ **Предусмотрены плейсхолдеры (placeholder) для полей.**

□ Если названия полей не подписаны, то внутри полей выводится подсказка, которая исчезает при внесении текста.

□ **Прописан атрибут autocomplete для полей, поддерживающих это значение.**

Атрибут autocomplete подставляет ранее введенные пользователем данные в поле, если функция не отключена в браузере.

□ **Правильная работа многошаговых форм.**

□ Навигация рядом с формой показывает текущий этап и количество оставшихся шагов.

□ Многошаговые формы корректно работают при навигации посредством кнопок «Вперед» и «Назад» в браузере.

Валидация

□ **Для числовых значений из определенного диапазона прописаны ограничители минимального и максимального количества символов**.

□ Проверьте это для дат, времени и прочих подобных характеристик.

□ **Для полей, предполагающих загрузку файлов, прописан атрибут accept, определяющий тип загружаемых документов.**

□ **Для полей, валидация которых проходит через регулярное выражение, прописан атрибут pattern.**

□ Требуемый формат данных, которые должен ввести пользователь, очевиден для него.

□ Доступна инструкция по формату вводимых данных на человеческом языке.

□ Пользователь не видит регулярного выражения как подсказки к действию.

□ **Сообщения об ошибках понятны обычным пользователям и логичны.**

Прочее

□ **Форма запрашивает у пользователя только необходимые данные.**

Откройте форму, визуально убедитесь, что требуется внести только необходимый минимум информации.

□ **Если все поля обязательны для заполнения, рядом с их названиями не выводятся звездочки — символ *.**

Откройте форму и убедитесь в этом визуально. Желательно наличие поясняющего текста об обязательном заполнении всех полей.

□ **Для авторизованного пользователя в поля формы автоматически подставляются все известные о посетителе данные.**

Убедитесь визуально, что указанная пользователем в профиле информация автоматически выводится в полях форм, запрашивающих эти данные.

□ **Текстовое многострочное поле при вводе объемного сообщения изменяет высоту либо в правой части появляется скроллбар для просмотра всего содержимого.**

Откройте форму с текстовым многострочным полем, введите в него максимально большое количество символов.

□ **В полях формы прописан корректный атрибут TYPE, сообщающий браузеру тип элементов формы.**

□ Правильно указаны типы дат, времени, телефонов, диапазонов, url, e-mail, чисел.

□ **Во время отправки формы на медленном канале пользователь не может менять в ней данные.**

Важно. Действительно для ajax-форм.

Low

□ **Вывод подсказок и ошибок сделан с анимационным эффектом.** Опционально и зависит от проекта.

□ **Кнопка отправки данных неактивна, пока не активирован чекбокс «Согласиться с правилами», «Пользовательское соглашение».**

□ **Кнопка отправки данных неактивна, пока все введенные данные не прошли положительную валидацию.**

Откройте форму с полями для ввода, введите некорректные данные, проверьте, активна ли кнопка.

□ **Если данные не прошли положительную валидацию, при наведении курсора на кнопку для отправки данных выводится информационное сообщение.**

Откройте форму, введите некорректные данные, наведите курсор на кнопку отправки данных, проверьте, выводится ли сообщение.

Задание.

Составьте детализированный чек-лист для тестирования этой формы. Ваша задача – не просто перечислить очевидные проверки, но и предусмотреть сценарии, которые часто упускают из виду.

Требования к чек-листу:

Структура: Разделите чек-лист на логические разделы (например: Валидация полей, Бизнес-логика, Работа с файлами, UX/Интерфейс, Безопасность и данные).

Приоритеты: Отметьте для каждого пункта приоритет (например, P0 — критично/блокирующее, P1 — важно, P2 — нормально).

Конкретность: Избегайте общих фраз. Вместо "Проверить валидацию email" пишите "При вводе 'user@domain' (без .com) поле подсвечивается красным, появляется текст ошибки 'Введите корректный email'".

Неочевидные сценарии: Включите минимум 5 пунктов, которые выходят за рамки базовой проверки "заполнил-отправил". См. подсказку ниже.

Подсказка: Какие неочевидные сценарии стоит рассмотреть?

Подумайте над этими аспектами, чтобы показать глубину тестового мышления:

- Логические зависимости: Ученая степень "Доктор наук" и возраст 22 года — это корректно?
- Работа с данными: Что происходит при отправке формы, если пока она грузилась, дедлайн на подачу заявок истек?
- Поведение при редактировании: Пользователь загрузил файл .pdf, затем передумал и выбрал .docx — старый файл должен заменяться?
- Граничные значения и форматирование: Ввод суммы 1 500 050 (шаг 50к) или 1 499 999. Ввод даты 31.02.2024.
- Интеграция: После успешной отправки формы проверьте, пришло ли письмо-подтверждение на email и корректно ли отобразилась заявка в личном кабинете (если он есть).

– Восстановление сессии: Заполнили 90% формы, случайно закрыли вкладку, открыли заново (если предусмотрено автосохранение черновика).

Пример начала чек-листа (для иллюстрации)

Чек-лист для формы "Подать заявку на грант"

Версия формы: 1.0

Приоритеты: P0 – Критичный, P1 – Высокий, P2 – Средний

Раздел 1: Валидация полей и бизнес-правила

[P0] Поле "ФИО": Отправка пустого поля вызывает сообщение "Это поле обязательно для заполнения".

[P1] Поле "Дата рождения": Ввод даты, делающей возраст 17 лет, вызывает ошибку "Возраст участника должен быть от 18 до 35 лет".

[P0] Поле "Email": Ввод "test@test" вызывает инлайн-валидацию с ошибкой до отправки формы.

Лабораторная работа 9

Тестирование строки поиска

Что надо проверить, тестируя поиск:

1. Поиск ищет по всем полям, указанным в ТЗ
2. Поиск НЕ ищет по тем полям, которые НЕ указаны в ТЗ
3. Релевантность выдачи — то, что я ищу, в начале списка, или в конце?
4. Учитывается ли контекст поиска — ищу я по всему сайту или только разделу игрушек
5. Регистронезависимость поиска — найдет ли «Платье», если я ввела «платье»?
6. Ищет ли по включению или полному соответствию — «ту» найдет мне «туфли»?
7. Найдет ли 2 слова из одного поля? В любом порядке введенные?
8. Найдет ли 2 слова из разных полей?
9. Ошибка в вводе (исправляются ли опечатки, ищет ли похожее)

10. Исправляет ли система неправильную раскладку?

11. Ищет ли на разных языках? А если сразу на двух попробовать?

12. Поиск со спецсимволами работает?

13. А с эмоджи? Не упадет система при вводе смайлика?

14. Принимаются ли открывающие и закрывающие пробелы

15. Пустое поле / только пробелы в поле

16. Нижнюю границу (от скольких символов ищет)

17. Верхнюю произвольную границу — указанную в ТЗ

18. Верхнюю границу на выходе

19. Поиск технологической границы — ввести «войну и мир», 100 млн символов.

Рассмотрим отдельно каждый пункт и выясним, как и зачем его проверять.

1. Поиск ищет по всем полям, указанным в ТЗ

Сначала надо проверить основное — то, что поиск вообще работает. Что он ищет по всем тем полям, по которым должен.

В любом чек-листе надо думать о приоритетах. Сначала — самое важное. Как в чек-листе в целом, так и внутри каждого блока проверок, постепенно идем от важного к неважному. От позитива к негативу.

А что самое важное в поиске? Для этого думаем, зачем его вообще делают. Чтобы искать. По чему? По каким-то полям / признакам. По каким? Узнаем и проверяем, ищет он по ним или нет.

2. Поиск НЕ ищет по тем полям, которые НЕ указаны в ТЗ

Необходимо убедиться, что поиск НЕ ищет там, где не должен.

3. Релевантность выдачи

Здесь важно проверить приоритет поисковой выдачи. Понятно, что туда может попасть что-то «не то». Чем больше у нас данных и чем больше полей, по которым поиск возможен, тем больше вариантов на каждый запрос.

4. Контекст поиска

Откуда я вызывается поиск? С главной страницы сайта или из конкретного раздела?

В интернет-магазинах обычно куча разных товаров, поэтому контекст поиска очень важен, чтобы выдавать релевантные товары.

А ещё контекст очень важен в мессенджерах. Искать вообще везде, во всех 100500 чатах, или только в одном?

.

5. Регистронезависимость поиска

Обычно поиск регистронезависимый, и это логично. Так что проверяем:

1. нижний регистр
2. ВЕРХНИЙ РЕГИСТР
3. ЗоЛоТую СеРеДиНу

6. Ищет ли по включению или полному соответствию

Поиск по включению — это когда можно ввести только часть слова. Например, «ко» вместо «котик». Это очень удобно во всяких чатах. Если поиск работает по полному совпадению, то нужно перебирать все падежи. Если он работает по включению, мне достаточно написать только часть слова.

Необходимо уточнить — как должно работать? А потом проверить:

- Поиск по полному соответствию в одном слове
- Поиск по частичному соответствию в одном слове — совпадение в начале слова / середине / конце (вспоминаем про классы эквивалентности и граничные значения)

7. Два слова из одного поля

А что будет, если у нас не одно слово, а два или более? Причем 2 слова у нас может быть:

1. В искомом поле
2. В поисковом запросе

И это будут разные тесты! Например, название товара: «Игровой набор». Тесты при этом:

1. Игровой → то есть в поле несколько слов, а мы ввели одно из них, найдётся?
2. Игровой набор → в поиске тоже несколько слов

Но что будет, если в поиске мы изменим порядок слов?

- Набор игровой → найдется ли он в таком формате? Или нужно прям четкое совпадение?

8. Два слова из разных полей

Если мы проверяем несколько слов в поиске, то тут тоже возможны варианты:

1. Они все из разных полей — например, мы ищем по цвету + названию + полу: «красная майка женская».
2. Они из одного поля — как выше с игровым набором. И в этом случае намного интереснее изменить порядок слов и проверить поведение системы =)

Важно понимать, что это разные тесты. И стоит проверить и тот вариант, и другой.

9. Опечатки

Как система работает с опечатками? Найдет ли похожее слово?

Краный галстук → Красный галстук

Если система работает с опечатками, то как:

- 1 неправильную букву исправит / 2 и более
- 1 пропущенную букву исправит / 2 и более

Это, конечно, зависит от длины искомого слова, но тогда исправляются ли опечатки в коротких словах?

10. Неправильная раскладка

Типичный пример опечатки — пользователь забыл изменить раскладку на клавиатуре и напечатал английскими символами русский текст. Ищем «котик» но вводим «rjnbr».

Если система русскоязычная и пользователь ввел данные на английском, по которым ничего не ищется, надо проверить — не забыл ли он переключить раскладку? Если по переключенной раскладке поиск работает, показываем эти результаты.

11. Другой язык

Что, если в системе есть данные и на русском, и на английском? Или даже смешанный вариант: «Сухой корм Purina ONE». Найдет ли система и по русскому алфавиту, и по английскому?

12. Спецсимволы

Это стандартный подход пуля для всех текстовых полей:

- Русский алфавит
- Английский
- Спецсимволы
- Эмоджи
- Перемешал

И поэтому приоритет у таких проверок неважный.

1. Спецсимвол есть в искомом поле.

Поэтому проверить нужно все спецсимволы. Можно: создать товар / искомый объект сразу с набором спецсимволов:

~!@#$%^&*()_+{}|:”>?<Ё!”№;%:?*()_+/Ъ,/.,;’[]\|

2. Пользователь может забыть переключить раскладку и вот уже вместо «Хлеб» он вводит «{kt,»

3. Пользователь может случайно вставить в поле какой-то текст со спецсимволами. Эти ошибки тоже надо предусмотреть.

13. Эмоджи

Эмоджи это не совсем спецсимвол. Система может не искать по спецсимволам, но при этом обрабатывать эмоджи. Следовательно, классы эквивалентности разные.

14. Принимаются ли открывающие и закрывающие пробелы

Метод trim() удаляет пробельные символы с начала и конца строки.

То есть вводим « тест », а система преобразует строку в «тест» и потом уже передаёт дальше, в нашем случае — функции поиска.

Проверка на трим более логична при заполнении и сохранении полей — введено случайно « Ольга» в имя, а потом поиск не находит, требуя точного совпадения. И тогда логично системе перед сохранением сделать трим.

15. Пустое поле

Фактически это проверка на ноль. А ноль — это отдельный класс эквивалентности, который часто приносит баги, поэтому его надо проверять.

Если мы оставили поисковую строку пустой, то есть варианты:

- Система выводит всю базу
- Система выводит пустоту
- Кнопка поиска просто не срабатывает / вообще заблокирована до ввода символов (что сомнительно, впрочем, да и зачем?)

16. Пробелы в поле

Ещё один вариант тестирования нуля — ввести в поле ТОЛЬКО пробелы. Как система их обработает? В данном тесте мы просто собираем информацию о том, как работает система. Потому что почти любое поведение (разве что кроме ошибки) можно считать нормальным.

17. Нижняя граница

Пустое поле (ноль) — мы уже проверили. А дальше думаем, какая у наших данных будет нижняя граница?

Павел Абдюшев приводит замечательный пример с писателем и поэтом Эдгаром ПО. Это пример короткой реальной фамилии, по которой могут искать. Попробуем найти там книгу «Ворон».

Сначала пробуем по фамилии:

ПО

Если книги автора найти не получается — то проблемы с приоритетами в выборке. Поиск по ФИО автора должен быть более релевантным, чем совпадение какого-то слова в описании.

18. Верхняя произвольная граница

Есть ли ограничение в строке поиска? Если есть, то оно обычно в разумных пределах — 100 / 500 / 1000 символов. И если пользователь вводит больше, то значение обрезается до максимального.

Иногда верхнюю границу просто не ставят. И это тоже нормально, это не баг, который надо срочно исправить. Просто верхней произвольной границы у нас не будет.

19. Верхняя граница на выходе

Прошлый пункт — о том, как подать много на вход. А что будет, если «много» не на входе, а на выходе? Или «где-то посередине», то есть там, где поиск идёт?

- Вернулось много данных по поиску (распространенный запрос)
- Много данных находится в самой системе / базе данных

Сколько времени займет поиск? И пройдет ли он вообще? Может на поиск установлен тайм-аут в 1 минуту. И при плохом интернете / большом объеме данных будут проблемы.

20. Поиск технологической границы

Для поиска технологической границы мы вбиваем в строку поиска ОЧЕНЬ БОЛЬШОЕ значение. 1000 символов — не поиск технологической границы. Введите 100 миллионов символов или главу «Войны и мира».

Задание.

Всестороннее тестирование строки поиска в интернет-магазине электроники

Контекст: Вы — тестировщик в компании "Gadget World", которая запускает новую версию сайта. Ваша задача — провести исчерпывающее тестирование основной строки поиска на главной странице. Поиск должен находить товары в каталоге магазина (смартфоны, ноутбуки, наушники и т.д.).

Базовая спецификация:

- Расположение: Центр хедера главной страницы.
- Элементы: Поле ввода + кнопка "Найти" (лупа).

- Поведение: Автодополнение (suggest) появляется после 3-го введенного символа. Поиск выполняется либо по нажатию кнопки, либо клавишей Enter.
- Особенность: Поиск работает по названию товара, бренду и основным характеристикам (например, по запросу "16 ГБ" могут находиться ноутбуки и смартфоны с такой памятью).

Задание

Ваша задача — составить максимально полный и структурированный чек-лист для тестирования этой строки поиска. Вам необходимо учесть функциональные, UX, перформанс и кросс-браузерные аспекты.

Требования к чек-листу:

1. Обязательные разделы:
   - Функциональность и логика поиска
   - Автодополнение (Suggest/Dropdown)
   - Валидация и обработка ввода
   - Удобство использования (UX)
   - Отображение результатов
   - Производительность
   - Кросс-браузерное и кросс-платформенное тестирование
   - Негативные и граничные сценарии
2. Включите не менее 5 "хитрых" сценариев, которые часто пропускают. Примеры:
   - Поиск товара, который только что закончился (статус "Нет в наличии").
   - Поведение при стремительном изменении запроса (пользователь быстро печатает и стирает).
   - Поиск с учетом регистра (iPhone vs iphone vs IPHONE).
   - Ввод XSS-инъекции (<script>alert('xss')</script>) в строку поиска.
   - SQL-инъекция (например, ' OR '1'='1).

- Поиск во время медленного соединения (не должен блокировать интерфейс).
- Копирование и вставка (Ctrl+C/Ctrl+V) длинного текста в поле.

Лабораторная работа 10

Комплексное тестирование программного продукта.

Тестирование, как способ контроля качества ПО

Тестирование – это проверка соответствия между реальным и ожидаемым поведением программы в специально заданных условиях. Исходной информацией для тестирования является сведения о том, как программная система должна себя вести, то есть требования к ней или к ее отдельной части. Наиболее распространенным способом тестирования программ является метод черного ящика. При этом реализация системы недоступна тестировщикам, и тестируется только ее интерфейс посредством пользовательского воздействия. Противоположностью данному методу является тестирование методом белого ящика, когда исходный код доступен тестировщикам и используется в качестве источника информации о тестируемой системе. В этом случае на основе требований к системе помимо ее реализации создается (программируется) ее тестовая модель.

По способу реализации тесты могут быть ручными и автоматическими. *Ручной тест* – это последовательность действий тестировщика (или разработчика), которую он может воспроизвести и при этом ошибка произойдет. Как правило, в средствах контроля ошибок такие последовательности действий содержатся в некоторой форме описания ошибки. *Автоматический тест* – это некоторая программа, которая воздействует на систему и проверяет то или иное ее свойство (функциональное требование).

Критерии тестирования

В любом случае качество тестирования может оказаться неудовлетворительным – ошибки находятся редко или вообще не находятся ввиду того, что количество возможных состояний программы очень велико, и

тестирование не в состоянии покрыть их все. На практике в реальных проектах, определяют критерии тестирования, которые определяют тот уровень качества, который необходимо обеспечить в данном проекте. Хорошее качество стоит дорого и очевидно, что разное ПО имеет разное качество, например, система управления химическим реактором и текстовый редактор. В силу ограниченности ресурсов на тестирование также целесообразно определить те аспекты ПО, которые наиболее важны как для общей работоспособности системы, так и для удовлетворения заказчика. При тестировании на некоторых (но не на всех возможных) входных данных применяют принцип факторизации – множество всех возможных входных значений разбивается на значимые с точки зрения тестирования классы и "прогоняются" тесты не на всех возможных входных значениях, но берут по одному набору значений из каждого класса. Например, тестируют некоторую функцию системы на ее граничные значения – очень большие значения параметров, очень маленькие и т.п. Часто факторизацию удобно делать, исходя из требований к данной функции. Также бывает полезно посмотреть на ее реализацию и протестировать ее по разным логическим веткам (порождаемым, например, условными операторами).

Задание

Рассматривая разработанный при выполнении курсовой работы или ВКР программный продукт как объект тестирования, выполнить тестирование в соответствии с заданием из таблицы 1.

Таблица 1. Задания на тестирование ПП

Автоматическое тестирование	*Ручное тестирование*
Метод белого ящика	
1. На основе функциональных требований, сформулированных при выполнении курсовой работы или ВКР, разработать несколько модульных тестов. Выполнить тесты. По проваленным тестам составить отчеты о	3. На основе перечислений альтернативных потоков событий, разработанных при выполнении курсовой работы или ВКР, выполнить стрессовое тестирование программного продукта. В случае обнаружения

несоответствии	некорректных ситуаций описать их, выявить причины и принять меры к их устранению
Метод черного ящика	
2. Выполнить нагрузочное тестирование ПП и оценить эффективность разработанных при выполнении курсовой работы или ВКР запросов к БД либо алгоритмов обработки данных исходных датасетов. Предварительно подготовить 4 набора данных на 10, 100, 1000 и 10000 записей. Построить графики зависимости времени вычислений от объема исходных данных	4. На основе перечислений основных потоков событий, разработанных при выполнении курсовой работы или ВКР, выполнить полную ручную проверку работоспособности ПП посредством воздействия на ее интерфейсную часть

При возникновении несоответствий задокументировать их, идентифицировать логические ошибки, при необходимости разработать дополнительные модульные тесты для их обнаружения. Устранить ошибки и провести рефакторинг.

Требования к содержанию отчета

В подразделе 1 приводятся результаты тестирования методом черного ящика. Приводятся описания и тексты программ модульных тестов и результаты их выполнения. Для ручных тестов приводятся алгоритмы проверки работоспособности ПП по всем возможным ветвям. Для этого требуется инициировать срабатывание всех пунктов меню в определенном порядке.

Представить обнаруженные логические ошибки (при наличии таковых). Для каждой логической ошибки:

− описать ее суть;

− представить тестовую последовательность манипуляций для ее обнаружения (как правило, это подмножество пунктов полной тестовой последовательности);

− представить форму ее проявления в результате применения теста;

− определить форму ее выражения в коде;

− описать способ ее устранения.

В подразделе 2 приводятся результаты тестирования методом белого ящика. Необходимо представить технические характеристики ПК, используемого для тестирования. Привести определенную последовательность действий пользователя (наподобие последовательности проверочных действий в подразделе 1). Также представить характеристики подготовленных наборов данных для нагрузочного тестирования. Указать источник данных и данные об объеме: число строк, слов, символов. При подготовке больших наборов данных допускается использование средств генеративного ИИ.

Привести временны́е и пространственные оценки проверок для каждой конфигурации и кратко их описать. Оценки нагрузочных проверок могут быть представлены в виде сводных таблиц и графиков на каждом наборе данных для каждого тестируемого метода. Описать результаты выполненного анализа. Результаты стрессового тестирования описать со ссылками на шифры альтернативных потоков событий, содержащих обработку некорректных ситуаций, причинами и способами их устранения.

Дополнительно:

Указывается *основной поток* событий для каждой автоматизированной функции ПП при нормальном его функционировании без учета возможных проблем, ошибок и некорректных ситуаций. Поток событий выполняется по пунктам нумерованным списком для каждой функции в отдельности. При этом следует обеспечивать максимально полное соответствие данного потока диаграмме требований UML, а также диаграммам A0 и A2.

По аналогичному принципу, но с учетом возможных проблем и некорректных ситуаций, разрабатываются *альтернативные потоки*. А для

обработки возможных ошибок при реализации основных и альтернативных потоков разрабатываются потоки ошибок.